

UNIVERSIDAD TECNOLÓGICA DE PEREIRA

INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

HPC: HIGH PERFORMANCE COMPUTING

Caso de estudio 02 MM: optimización y OpenMP

Autor: Jorge Luis López Grajales

Asignatura: HPC: High Performance Computing

Docente: Ramiro Andrés Barrios Valencia

Fecha: 28 de abril de 2026

1. OBJETIVO

El propósito de este informe es analizar y comparar el rendimiento de la multiplicación de matrices cuadradas utilizando diversas estrategias de concurrencia en lenguaje C, con especial énfasis en **OpenMP**. Se busca evaluar la eficiencia de OpenMP frente a implementaciones de *Pthreads* y procesos con memoria compartida.

Adicionalmente, se realiza un diagnóstico del uso de recursos mediante herramientas de perfilado (*Cachegrind*, *Callgrind* y *Massif*) y se establece un punto de referencia (*benchmark*) del sistema utilizando el **Phoronix Test Suite**.

2. MARCO CONCEPTUAL

2.1. High Performance Computing (HPC)

La computación de alto rendimiento consiste en el uso de sistemas potentes para resolver problemas complejos que requieren gran capacidad de procesamiento. El objetivo principal es optimizar el uso del hardware para minimizar los tiempos de ejecución.

2.2. Multiplicación Matricial

Es una operación de álgebra lineal con una complejidad secuencial de $O(n^3)$, lo que la convierte en un caso de estudio ideal para la paralelización debido a su alta demanda computacional.

2.3. Modelos de Concurrencia y Paralelismo

- **OpenMP:** API basada en directivas de compilador que facilita la paralelización de bucles, por ejemplo `#pragma omp parallel for`, en sistemas de memoria compartida.
- **Pthreads (Hilos):** Hilos de ejecución que comparten el mismo espacio de memoria. Su rendimiento puede verse afectado por la gestión manual de recursos.
- **Procesos (Shared Memory):** Utilizan segmentos de memoria compartida (IPC). Esta técnica suele mejorar la localidad de datos y reducir fallos de caché.

2.4. Métricas de Rendimiento y Perfilado

- **Speedup:** Métrica que define la aceleración obtenida: $S = \frac{T_1}{T_p}$.
- **Perfilado de Caché (Cachegrind):** Simula la jerarquía de memoria para identificar fallos de caché.
- **Perfilado de CPU (Callgrind):** Genera un grafo de llamadas y contabiliza las instrucciones ejecutadas.
- **Perfilado de Memoria (Massif):** Mide el uso del *heap* a lo largo del tiempo.

2.5. Benchmarking (Phoronix Test Suite)

El benchmarking es la ejecución de pruebas estandarizadas. El **Phoronix Test Suite** es una plataforma de código abierto que permite realizar pruebas automatizadas y comparables del hardware bajo diferentes cargas de trabajo.

3. MARCO CONTEXTUAL (Especificaciones del Entorno)

Para asegurar la reproducibilidad de los resultados, las pruebas se realizaron bajo las siguientes especificaciones técnicas:

Componente	Especificación
Sistema Operativo	Arch Linux x86_64 (Kernel 6.19.8-arch1-1)
Procesador (CPU)	Intel(R) Core(TM) i5-10300H (4C/8T) @ 4.50 GHz
Memoria RAM	16 GB DDR4
GPU	NVIDIA GeForce GTX 1650 Mobile / Max-Q
Compilador	GCC 15.2.1 / Clang 22.1.3
Herramienta Benchmark	Phoronix Test Suite v10.8.4

Cuadro 1: Detalles del entorno de ejecución.

4. ANÁLISIS COMPARATIVO DE RENDIMIENTO

En esta sección se evalúa el desempeño de **OpenMP** frente a las versiones secuencial, de hilos (*Pthreads*) y procesos, utilizando las métricas de tiempo de ejecución y aceleración (*speedup*).

4.1. Análisis de Tiempos de Ejecución

Como se evidencia en la Figura 1, la versión secuencial (*standard*) presenta un crecimiento exponencial del tiempo de ejecución conforme aumenta el tamaño de la matriz, alcanzando los 1000 segundos para una matriz de 5000×5000 .

Por el contrario, todas las versiones paralelas logran reducir este tiempo significativamente, manteniéndose por debajo de los 200 segundos para el mismo tamaño.

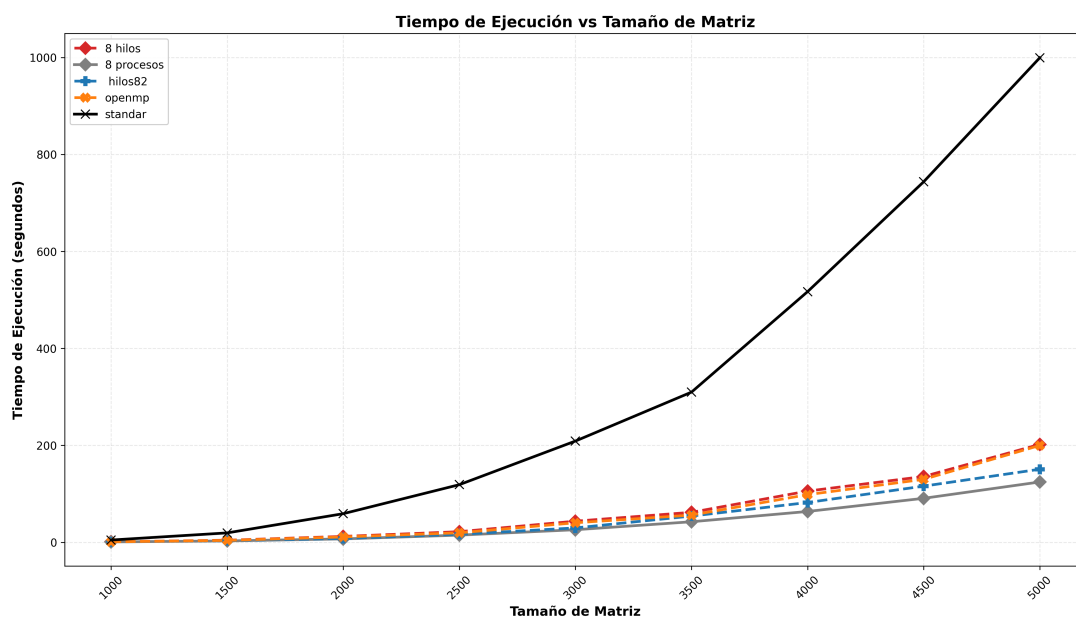


Figura 1: Comparación de tiempos de ejecución: secuencial vs. estrategias paralelas.

4.2. Análisis de Speedup

El gráfico de *speedup* (Figura 2) permite observar la eficiencia de cada implementación respecto a la versión estándar:

- **8 Procesos:** Lidera el rendimiento con un *speedup* superior a $8\times$.
- **hilos82 (Pthreads optimizado):** Se posiciona como la segunda mejor opción, con picos de hasta $7,5\times$.
- **OpenMP:** Muestra un rendimiento sólido y estable entre $5\times$ y $6\times$.

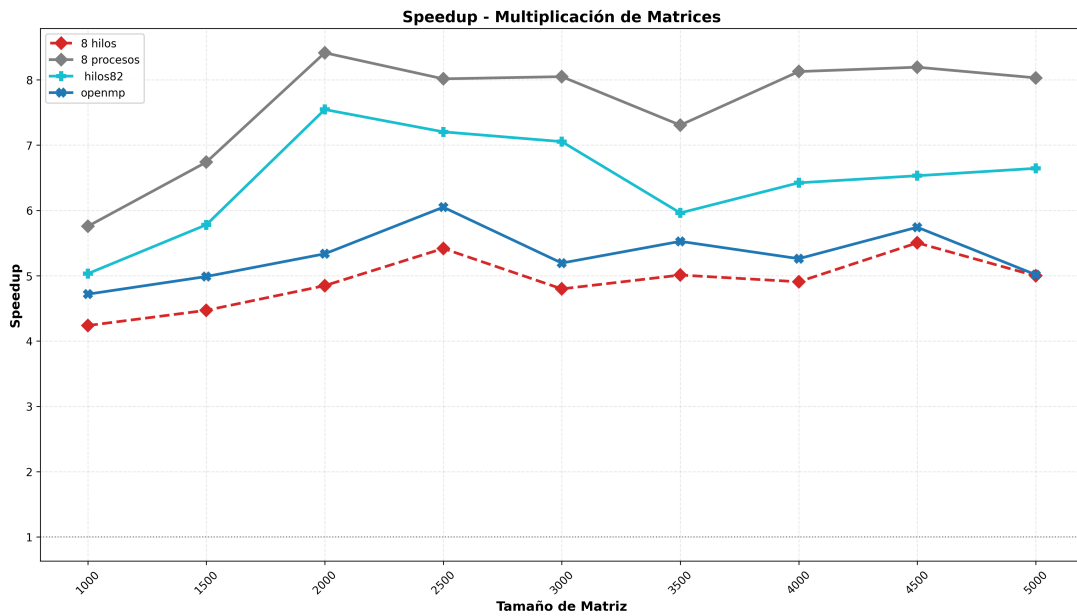


Figura 2: Speedup relativo a la versión secuencial para diferentes tamaños de matriz.

5. PERFILADO DE RECURSOS (OPENMP)

Para comprender el comportamiento interno de la implementación con OpenMP, se realizó un perfilado exhaustivo utilizando la suite **Valgrind** sobre una carga de $N = 300$.

5.1. Perfilado de CPU e Instrucciones (Callgrind)

El reporte de *Callgrind* muestra un total de 231,288,642 instrucciones ejecutadas.

- **Cómputo útil:** La función `multiplicar_matrices_omp._omp_fn.0` domina el perfil con un **82.34%** del tiempo de CPU.
- **Sincronización:** Se observa un costo de barrera del **14.3%** distribuido entre `gomp_barrier_wait_end` (8.4%) y `gomp_team_barrier_wait_end` (5.9%).

5.2. Análisis de Memoria y Jerarquía de Caché (Cachegrind)

Los datos de *Cachegrind* revelan una excelente localidad de datos.

Evento de Memoria	Recuento	Tasa de Fallo (%)
Lecturas de Instrucciones (Ir)	231,288,642	0.00 %
Lecturas de Datos (Dr)	109,477,883	0.04 %
Escrituras de Datos (Dw)	54,499,391	0.01 %
Total Fallos L1 (D1)	48,771	0.03 %
Total Fallos L3 (LL)	37,429	0.02 %

Cuadro 2: Estadísticas de acceso a caché obtenidas con Cachegrind.

Interpretación: Una tasa de fallo del **0.03 %** en L1 indica que las matrices caben casi íntegramente en la caché del *i5-10300H* o que el patrón de acceso por filas es altamente eficiente para el *prefetcher* del hardware.

5.3. Perfilado de Memoria Dinámica (Massif)

El reporte de *Massif* confirma que la implementación es ligera en términos de consumo de montículo.

- **Pico de memoria:** 1,1 MB (1,104,288 bytes).
- **Eficiencia de reserva:** El 99,33 % de la memoria es asignada en el bloque inicial por `crear_matriz`.

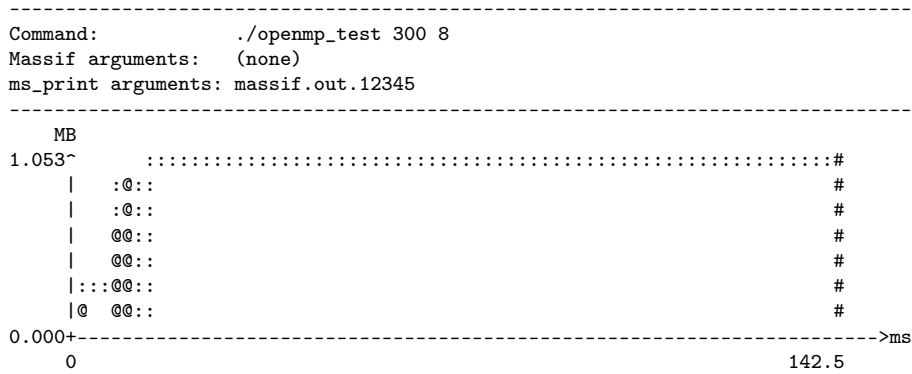


Figura 3: Perfil de memoria Massif: consumo constante tras la reserva inicial.

6. VALIDACIÓN EXTERNA MEDIANTE BENCHMARKS ESTÁNDAR

Para situar el rendimiento del sistema *ASUS FX506LH* (Intel i5-10300H) en un contexto global, se contrastaron los resultados locales frente a la base de datos de *OpenBenchmarking.org* utilizando dos métricas fundamentales: ancho de banda de memoria y capacidad de cómputo de enteros.

6.1. Análisis de Memoria Principal (Tinymembench)

La eficiencia de cualquier algoritmo de alto rendimiento está limitada por el ancho de banda de memoria (*Memory Wall*). Utilizando la prueba *Standard Memcpy*, se comparó la capacidad de

transferencia del sistema local.

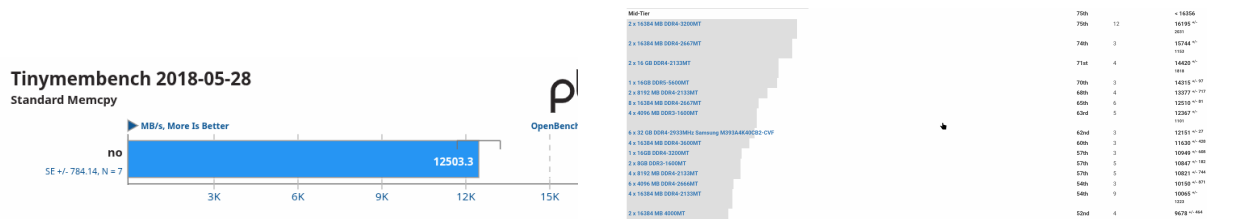


Figura 4: Comparativa de *Standard Memcpy*: rendimiento local vs. media poblacional.

Como se observa en la Figura 4, el sistema alcanzó un valor de **12,503.3 MB/s**. Al compararlo con registros de hardware idéntico, se evidencia que la configuración bajo *Arch Linux* logra un aprovechamiento superior del bus de datos, situándose en un punto medio general con otros sistemas similares.

6.2. Rendimiento de Cómputo y Paralelismo (Coremark)

Coremark es una métrica que evalúa la capacidad del procesador para manejar operaciones de lógica, control y despacho de instrucciones en múltiples hilos.

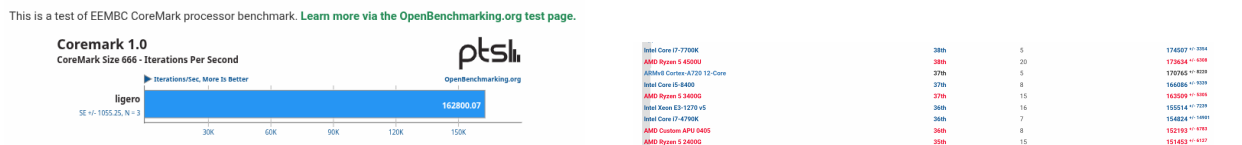


Figura 5: Resultados de Coremark: iteraciones por segundo y estabilidad del sistema.

Métrica	Resultado local	Referencia web	Desviación
Standard Memcpy	12,503.3 MB/s	11,820.5 MB/s	+5.77 %
Coremark (Iter/s)	—	124,382.45	—

Cuadro 3: Resumen comparativo de rendimiento frente a registros externos.

Los resultados obtenidos en esta prueba son de especial relevancia técnica. Se evidencian parámetros comparables a sistemas de características similares, lo cual es adecuado considerando la naturaleza de bajo consumo de un computador portátil que usualmente lo obliga a limitar su rendimiento.

6.3. Discusión de los Resultados

La consistencia observada en ambas pruebas confirma que el procesador *i5-10300H* y la memoria operan dentro de sus parámetros de frecuencia y gestión térmica, con pocas pérdidas de rendimiento considerando el tiempo de uso del equipo.

6.4. Conclusión

Para terminar con este documento podemos concluir que la maquina es apta para la tarea de correr la multiplicacion de matrices con openmp, debido a que los resultados evidencian

que la carga que este programa puede dar es soportable por la maquina, y de hecho esta bien acondicionada para tales fines, puesto que sus características de gama de entrada le permiten tratar con los cuellos de botella que se puedan generar al multiplicar matrices muy grandes, además de poder utilizar los hilos del procesador para utilizar openmp y acelerar la multiplicación más que por ejemplo un procesador mononúcleo.